Experiment 9(a) - Parallel Processing Using MPI

Step-by-Step Practice Guide for Ubuntu

Aim: To implement parallel processing using MPI in Linux.

What you will do: install Python, MPICH and mpi4py; create a Python MPI program; run three processes; and find the maximum value across distributed array chunks.

Quick setup checklist

• Ubuntu / Ubuntu Virtual Machine

• Internet connection for package installation

• Terminal access

• A text editor such as nano

Step 1 - Update Ubuntu

```
sudo apt update
```

Enter your Ubuntu password if asked and allow the package list to update.

Step 2 - Install Python 3

```
sudo apt install python3
```

Verify:

```
python3 --version
```

Step 3 - Install the MPI environment

```
sudo apt install -y -qq mpich
```

Verify that MPI is available:

```
mpirun --version
```

Step 4 - Install pip

```
sudo apt install python3-pip
```

Verify:

```
pip3 --version
```

Step 5 - Install mpi4py

Your record installs the Ubuntu package directly:

```
sudo apt install python3-mpi4py
```

Optional verification:

```
python3 -c "from mpi4py import MPI; print(MPI.Get_version())"
```

Step 6 - Create the MPI Python program

Create the file:

```
nano array_max.py
```

Type the following program from the record:

```
from mpi4py import MPI

com = MPI.COMM_WORLD
rank = com.Get_rank()
size = com.Get_size()

arr = [[10,20],[100,9],[200,4]] if rank == 0 else None
```

```
local = com.scatter(arr, root=0)

local_max = max(local)
print(f"process {rank} local max {local_max}")

global_max = com.reduce(local_max, op=MPI.MAX, root=0)

if rank == 0:
    print("Maximum value is: ", global_max)
```

Save in nano: **Ctrl+O**, press **Enter**, then **Ctrl+X**.

Step 7 - Run the program using 3 MPI processes

```
mpirun -np 3 python3 array_max.py
```

Expected result shown in your record:

```
process 0 local max 20
process 1 local max 100
process 2 local max 200
Maximum value is:  200
```

Practice note: In parallel programs, the order of the individual process print lines can vary between runs. The final maximum should still be 200.

What happens in the program?

• **MPI.COMM_WORLD** represents all MPI processes.

• **Get_rank()** gives each process its ID: 0, 1, 2.

• **scatter()** sends one sub-array to each process.

• **max(local)** finds the local maximum for each process.

• **reduce(..., MPI.MAX)** combines the local answers and gives the global maximum to process 0.

Quick exam rehearsal

```
sudo apt update
sudo apt install python3
sudo apt install -y -qq mpich
sudo apt install python3-pip
sudo apt install python3-mpi4py
nano array_max.py
mpirun -np 3 python3 array_max.py
```

Common mistakes

• Running with plain **python3 array_max.py** instead of **mpirun -np 3 ...**.

• Typing **mpi4py** imports incorrectly.

• Using a process count different from the three chunks in the source array during this exact exercise.

• Forgetting to save the file before running it.

Result

Thus the implementation of parallel processing using MPI in Linux is completed successfully.